

MILESTONE

Escrow-Backed, Multi-Freelancer Project Management for Mobile

Complete Build & Ship Plan

A step-by-step roadmap from Figma design to a shipped MVP, built using free AI tools (Google Antigravity, Gemini CLI, Claude, ChatGPT) — the "vibe coding" workflow.

Prepared for: Product & Design Founder (Figma / AI-native mobile apps)
Document type: Internal build plan — not legal, financial, or licensing advice

Contents

- 1. Executive Summary
- 2. Scope: MVP vs. Phase 2
- 3. Tech Stack for a Vibe-Coded Build
- 4. The Vibe Coding Workflow (How the 4 Tools Work Together)
- 5. Phase 0 — Design in Figma
- 6. Phase 1 — Environment & Project Setup
- 7. Phase 2 — Data Model & Backend Logic
- 8. Phase 3 — Core App Build (Screen by Screen)
- 9. Phase 4 — Simulated Escrow & Milestone Engine
- 10. Phase 5 — Identity Verification (KYC) Flow
- 11. Phase 6 — AI Features Layer
- 12. Phase 7 — Testing & QA
- 13. Phase 8 — Shipping (TestFlight / Play Internal / Web)
- 14. Full Timeline (8-Week Plan)
- 15. Prompt Library — Ready-to-Use Prompts per Tool
- 16. Compliance & Real-Money Reality Check
- 17. Post-Launch Checklist

1. Executive Summary

Milestone is a mobile app that lets clients fund a project's full budget upfront, split it into milestones with individual deadlines and payouts, and assign those milestones to one or more freelancers. Funds release automatically 72 hours after milestone submission unless the client reviews and approves sooner, and disputes route to a structured company review. Both sides verify identity at signup.

This document is a build plan for shipping a working MVP using free AI coding tools — **Google Antigravity**, **Gemini CLI**, **Claude**, and **ChatGPT** — in a "vibe coding" workflow: you direct the tools in plain language, review what they produce, and iterate, rather than hand-writing every line. The plan is organized as 8 build phases plus a prompt library so you can move straight from reading this to building.

Reality check up front: the escrow, KYC, and auto-release mechanics described here can be fully **simulated** in an MVP (fake wallets, mock verification, in-app state) so you can prove the product and UX end-to-end without a money-transmitter license. Moving real user funds requires a licensed payments partner — this is covered in Section 16.

2. Scope: MVP vs. Phase 2

MVP (what you actually build first, weeks 1–8)

- Signup/login for Client and Freelancer roles, with a **simulated** ID verification step
- Client creates a project: total budget, description, and 2–6 milestones (name, deadline, payout amount)
- Client funds the project (simulated wallet debit) before it becomes visible/postable
- Freelancers can be invited or apply per milestone (not per whole project)
- Milestone status pipeline: Not Started → In Progress → Submitted → Approved/Auto-Released → Paid
- 72-hour countdown on submitted milestones with simulated auto-release
- Per-contract chat (text + file attachment)
- Simulated escrow wallet screen: funded / held / released ledger per milestone
- Basic dispute flow: freelancer or client flags a milestone → status freezes → simple review queue

Phase 2 (after MVP validates)

- Real identity verification via a KYC provider (e.g., Persona, Stripe Identity)
- Real escrow via a licensed payments partner (see Section 16)
- Push notifications, ratings/reputation system, AI-assisted scope-vs-deliverable comparison
- Search/discovery, admin dashboard for dispute staff, multi-currency support

3. Tech Stack for a Vibe-Coded Build

Pick a stack that the AI tools already know extremely well — this matters more than picking the "best" stack, because AI-assisted output quality is highest on the most common patterns.

Layer	Tool	Why
Mobile app	React Native (Expo)	Cross-platform, huge training data for AI tools, fast preview via Expo Go — no native build tooling needed for the MVP
Backend / DB	Supabase (Postgres + Auth + Storage)	Free tier, generates types automatically, has a hosted auth system you can point "role: client/freelancer" at
State/simulated wallet	Postgres tables in Supabase	"Simulated" just means the numbers live in your own DB, not a real payment rail — still feels real to users
Chat	Supabase Realtime	Free, works out of the box with Postgres, no separate chat service needed for MVP
Hosting (backend)	Supabase Cloud (free tier)	No server to manage
App distribution	Expo EAS + TestFlight / Play Internal Testing	Free tiers cover MVP-scale testing
Design	Figma	Your existing tool — export specs/tokens straight into prompts for the AI tools

4. The Vibe Coding Workflow — How the 4 Tools Work Together

Each tool has a different strength. Using them in the wrong order is the #1 cause of vibe-coding projects turning into unmanageable spaghetti. Use this division of labor:

Tool	Primary Role	Use it for...
ChatGPT	Planning & spec writing	Turn this PDF's phases into a detailed feature spec, break features into small tasks, write copy/microcopy, sanity-check logic (e.g. "does this milestone state machine have gaps?")
Claude	Architecture & complex code	Design the database schema, write the milestone/escrow state-machine logic, review generated code for bugs, refactor messy files, write clear code comments
Gemini CLI	In-terminal execution	Run inside your project folder to scaffold files, install packages, run/fix build errors, do repetitive multi-file edits directly on disk
Google Antigravity	Agentic build & iteration	Give it a screen spec + Figma reference and let it build/iterate the actual React Native screen in a tighter loop, testing as it goes

The loop, repeated for every feature

- 1 **Spec it (ChatGPT):** Write a 5–10 line spec: what the feature does, inputs, outputs, edge cases.
- 2 **Design it (Claude):** If it touches data or state, ask Claude to define the schema/state machine first, before any UI code exists.
- 3 **Scaffold it (Gemini CLI):** From your terminal, ask Gemini CLI to create the files/folders and install any needed packages.
- 4 **Build it (Antigravity):** Point Antigravity at the spec + the Figma frame, let it write and iterate the screen until it matches.
- 5 **Review it (Claude):** Paste the generated code back to Claude for a bug/security/readability pass before moving on.
- 6 **Test it yourself:** Actually tap through the flow on a real device via Expo Go. Vibe coding still requires a human to check the vibe.

Golden rule: never let an AI tool touch more than one feature at a time. Small, reviewed diffs are what keep a vibe-coded app shippable instead of collapsing under its own complexity around week 3.

5. Phase 0 — Design in Figma (Week 1)

Design just enough to build from — not a full product. Aim for 10–14 screens, one primary state each.

- Auth: Role select → Signup → (Simulated) ID verification → Home
- Client: New Project wizard (budget → milestone split → deadlines) → Project Dashboard
- Freelancer: Browse/Invited Milestones → Milestone Detail → Submit Work
- Shared: Contract Detail (milestone timeline + escrow ledger), Chat Thread, Escrow Wallet, Dispute/Trust Review, Notifications

Deliverable to carry into build

Export a simple design-tokens sheet (colors, spacing, font sizes) as a short text file — you'll paste this directly into your first Antigravity/Claude prompts so generated screens match your Figma, not generic Bootstrap-style defaults.

6. Phase 1 — Environment & Project Setup (Day 1–2 of Week 2)

- 1 Install Node.js (LTS) and set up a free GitHub account + new private repo named **milestone-app**.
- 2 Install Expo CLI: **npx create-expo-app milestone-app**.
- 3 Create a free Supabase project → note the project URL and anon key.
- 4 In terminal, open Gemini CLI inside the project folder and ask it to install and configure the Supabase client, add a **.env** file, and create a basic folder structure: **/screens /components /lib /hooks**.
- 5 Commit this skeleton to GitHub before writing any feature code — this is your rollback point.

7. Phase 2 — Data Model & Backend Logic (Week 2)

Give this schema (or your Claude-refined version of it) to Supabase before any UI work starts.

Table	Key fields
users	id, role (client/freelancer), name, verification_status, created_at
projects	id, client_id, title, description, total_budget, status, created_at
milestones	id, project_id, freelancer_id, title, deadline, payout_amount, status, submitted_at
wallets	id, user_id, balance (simulated)
escrow_ledger	id, milestone_id, amount, state (funded/held/released), timestamp
messages	id, project_id, sender_id, body, attachment_url, created_at
disputes	id, milestone_id, opened_by, reason, evidence, status, resolved_at

Ask Claude specifically to design the **milestone status state machine** (valid transitions only — e.g. a milestone can't go from Not Started straight to Paid) and to write it as a small pure function you call from every place status changes. This single function is what prevents the auto-release logic from ever double-paying or skipping approval — it's the most important piece of logic in the whole app.

8. Phase 3 — Core App Build, Screen by Screen (Weeks 3–4)

Build in this order — each step unblocks the next and gives you something tappable to test daily.

#	Screen/Feature	Primary Tool	Notes
1	Auth + role select	Antigravity	Supabase Auth, store role on signup
2	Client: New Project wizard	Antigravity + Claude	Claude reviews the budget-split math before UI is trusted
3	Simulated wallet funding	Claude	Debits client wallet, writes escrow_ledger row as 'funded'
4	Project/Contract dashboard	Antigravity	Shows milestone timeline + % complete
5	Freelancer milestone view + submit	Antigravity	Upload deliverable, set status to Submitted
6	Approval + 72h auto-release	Claude	Timer/cron logic — see Phase 4 for detail
7	Chat thread	Gemini CLI + Antigravity	Supabase Realtime subscription per project
8	Escrow wallet/ledger screen	Antigravity	Reads escrow_ledger, groups by milestone
9	Dispute flow	Antigravity + Claude	Freezes milestone status, opens review record

9. Phase 4 — Simulated Escrow & Milestone Engine (Week 4–5)

This is the trust core of the product — worth its own focused pass after the base screens work.

- **Funding:** creating a project only succeeds if `wallet.balance ≥ total_budget`; on success, debit the client wallet and write one `escrow_ledger` row per milestone as 'held'.
- **Submission:** freelancer sets milestone status to Submitted and stores `submitted_at` — this timestamp starts the 72-hour clock.
- **Approval:** client action moves `escrow_ledger` row to 'released' and credits the freelancer's wallet immediately.
- **Auto-release:** a scheduled Supabase Edge Function (free tier) runs hourly, finds milestones Submitted > 72 hours ago with no client action, and auto-releases them the same way an approval would.
- **Dispute override:** opening a dispute must freeze the 72-hour clock — check this explicitly, it's the easiest edge case to miss.

Ask Gemini CLI to scaffold the Supabase Edge Function, then have Claude review the cron logic specifically for race conditions (e.g., client approves at the exact moment auto-release fires).

10. Phase 5 — Identity Verification Flow (Week 5)

For the MVP, simulate the flow so the UX is real even though the check isn't:

- Signup asks for name, DOB, and a photo ID upload (stored in Supabase Storage)
- Status field `verification_status` starts 'pending' → a simple mock rule (e.g., file uploaded + name filled) flips it to 'verified' after a short delay, to simulate review
- Users can't fund or accept a milestone while `verification_status` is 'pending'
- Design this screen so swapping in a real provider later (Section 16) only changes the backend call, not the UI

11. Phase 6 — AI Features Layer (Week 6)

Given your niche (designing for AI products), this is the layer worth polishing most — it's your differentiator.

- **Milestone auto-breakdown:** client pastes a project scope → call an LLM API (e.g., via a free-tier key) to propose a milestone split with suggested deadlines/payouts, which the client edits before confirming
- **Status-update drafts:** freelancer taps 'Draft update' in chat → LLM turns their rough notes into a clear client-facing message
- **Scope-creep flag:** when work is submitted, compare the deliverable description against the original milestone spec and flag mismatches for the client to review

Build these as clearly separate, optional calls (not baked into the core state machine) so the app works fully even if an AI call fails or a free API quota runs out.

12. Phase 7 — Testing & QA (Week 7)

Manual test pass (do this yourself, on a real phone via Expo Go)

- Full happy path: client creates + funds project → freelancer submits → client approves → wallet updates
- Auto-release path: submit a milestone, manually set submitted_at 73 hours in the past in Supabase, confirm the Edge Function releases it
- Dispute path: open a dispute mid-countdown, confirm the clock freezes and status doesn't auto-release
- Multi-freelancer project: two freelancers on one project, confirm dashboards and wallets stay isolated per person
- Edge cases: funding with insufficient balance, submitting after deadline, chat with attachments

AI-assisted QA

- Ask Claude to write a short test checklist from your actual schema/state machine and diff it against what you tested manually
- Ask ChatGPT to role-play as a confused first-time user reading your screens' copy and flag anything unclear

13. Phase 8 — Shipping (Week 8)

- 1 Run **eas build** (Expo Application Services, free tier) to produce iOS and Android builds.
- 2 Submit the iOS build to TestFlight for internal testing (free, requires a \$99/yr Apple Developer account only when you're ready for external testers).
- 3 Submit the Android build to Google Play Internal Testing track (free, requires a one-time \$25 Play Console fee).
- 4 Alternatively, ship a web build first via Expo's web export — zero app-store friction while you validate the flow with real users.
- 5 Write a short release note / changelog and a 1-page "how to test" doc for your first 5–10 testers.

14. Full Timeline — 8-Week Plan

Week	Focus
1	Figma design — all core screens + design tokens exported
2	Project setup + Supabase schema + auth working
3	Client project creation + wallet funding (simulated)
4	Freelancer submission flow + approval/escrow ledger UI
5	Auto-release engine + dispute flow + verification flow (simulated)
6	Chat + AI features layer (milestone breakdown, draft updates)
7	Full manual + AI-assisted QA pass, bug fixing
8	Builds, TestFlight/Play Internal submission, first tester round

This assumes solo, part-time-to-full-time work. Compress to 4–5 weeks if building full-time, or extend Phase 3 by a week if this is your first React Native/Supabase project.

15. Prompt Library — Ready-to-Use Prompts

ChatGPT — Spec writing

"Turn this feature into a numbered spec with inputs, outputs, and edge cases, written for a developer who will hand it to an AI coding tool: [paste feature from this plan]."

Claude — Schema & state machine

"Design a Postgres schema in Supabase for a milestone-based escrow app with these tables: [paste schema table above]. Then write a pure state-machine function for milestone status transitions that prevents invalid jumps (e.g. Not Started to Paid) and explain each guard clause."

Gemini CLI — Scaffolding

"In this project folder, create a Supabase Edge Function called auto-release-milestones that runs hourly, queries milestones with status Submitted and submitted_at older than 72 hours, and calls the shared releaseMilestone() function on each. Install any needed dependencies."

Google Antigravity — Screen build

"Build the Contract Detail screen in React Native/Expo matching this Figma frame: [attach/describe frame]. Use these design tokens: [paste tokens]. It should show a milestone timeline stepper, an escrow ledger list (funded/held/released), and a 72-hour countdown badge on any Submitted milestone. Pull data from Supabase tables projects, milestones, and escrow_ledger."

16. Compliance & Real-Money Reality Check

This section is informational, not legal advice — confirm specifics with a lawyer before handling real user funds.

- **Holding other people's money is regulated.** In most jurisdictions, an app that holds client funds and releases them to third parties is acting as a money transmitter / escrow agent, which typically requires licensing or partnering with an already-licensed provider.
- **The practical path:** use a licensed escrow-as-a-service or payments provider (e.g., providers built on Stripe Connect, or dedicated marketplace-escrow APIs) once you move past the simulated MVP — they hold the license so you don't have to.
- **Identity verification** for real money movement will similarly need a real KYC provider (e.g., Stripe Identity, Persona) rather than the simulated check used in the MVP.
- **What's safe to ship now:** the entire simulated version of this app — UI, flows, chat, fake wallet state — is safe to build and demo to real users to validate the product, as long as it's clearly not moving real money yet.

17. Post-Launch Checklist

- Collect structured feedback from first 10 testers on the project-creation and approval flows specifically — these are the highest-friction moments
- Track where users drop off in the milestone wizard (this is your #1 UX risk area)
- Decide on a real payments/KYC partner before opening the app beyond simulated testing
- Revisit the AI features layer — this is where you can differentiate fastest given your design background
- Start a simple changelog so early users can see the app improving week over week

End of document — Milestone Project Build Plan.